Relatório de Auditoria de Segurança — LibManager

Data do relatório: 20/09/2026
Repositório: https://github.com/VitorFranca089/LibManager

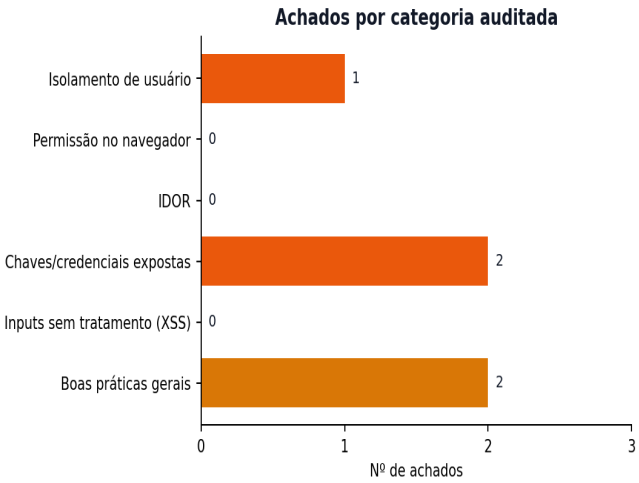
Escopo auditado	Código-fonte completo do repositório LibManager (branch main), incluindo camadas Controller, Service, DAO, Model/DTO, utilitários, migrações SQL (Flyway) e configuração de conexão com banco de dados. Auditoria estática de código-fonte com verificação dinâmica pontual (execução local em ambiente isolado com dados fictícios) para confirmar o achado principal.
Stack detectada	Java 21+ (compilado/testado com JDK 25), aplicação de linha de comando (sem frontend web), JDBC puro (sem ORM) via mysql-connector-j, MySQL 8.x, migrações de schema com Flyway. Autenticação própria (usuário/senha na tabela users), sem framework de autorização (ex.: Spring Security). Empacotamento Maven.
Metodologia	Aplicação das cinco categorias do prompt de auditoria de segurança, adaptadas à stack detectada: (1) isolamento de usuário mapeado para os filtros manuais por id_user nas queries JDBC, já que não há RLS nem middleware de tenant; (2) permissão definida no navegador mapeada para a fronteira entre menu/UI e camada de serviço — não aplicável por não existir separação cliente/servidor; (3) IDOR mapeado para todo handler que recebe um ID (usuário, livro, empréstimo) via entrada do usuário; (4) chaves/credenciais expostas revisado em código-fonte, migrações SQL e configuração; (5) XSS/inputs sem tratamento não aplicável por não existir renderização de HTML/navegador. Revisão arquivo por arquivo de todo o pacote io.github.vitorfranca089.libmanager, sem amostragem.
Grupo responsável	Vitor Miguel Mendes de França Daniel Carrapeiro Mendes Anita Felicio dos Santos Fábia Aurelie Losija

Finalidade educativa e responsável: este relatório documenta uma auditoria de segurança autorizada (projeto de autoria do próprio grupo), com o objetivo de ajudar a corrigir problemas reais e melhorar a segurança do sistema. Nenhum dado real de terceiros foi acessado, copiado ou alterado; toda a verificação dinâmica foi feita com contas e dados fictícios em ambiente local isolado, removidos ao final dos testes.

Resumo executivo

Foram identificados **5 achados** (Crítica: 1 · Alta: 2 · Baixa: 1 · Informativa: 1), cobrindo as cinco categorias do prompt de auditoria (duas não aplicáveis à stack, justificadas na seção correspondente) e 3 pontos fortes confirmados. Uma vulnerabilidade **Crítica** (F2) e uma **Alta** (F1) foram confirmadas com evidência de exploração — a F1 inclusive reproduzida ponta a ponta na aplicação real com dados fictícios.





Pontos fortes (controles verificados)

Ponto forte

Nenhuma ocorrência de SQL Injection

Todas as 17 consultas SQL do projeto (arquivos AuthDAO.java, UserDAO.java, BookDAO.java, LoanDAO.java) usam PreparedStatement com parâmetros via bind (setString/setInt/setTimestamp). Não há uso de Statement genérico nem concatenação de entrada do usuário diretamente em texto SQL — verificado por varredura em todo o pacote dao/.

Ponto forte

IDOR mitigado nos fluxos acessíveis ao usuário comum

Nenhum handler acessível ao papel COMMON aceita um ID de outro usuário informado por ele mesmo: LoanController.userLoanMenu() (linhas 108-116) e ProfileController.profileMenu() (linha 12, chamado com user.id() da sessão autenticada em MainController.java:36) sempre usam o objeto do usuário logado, nunca um ID arbitrário digitado. Todos os fluxos que recebem IDs de terceiros (UserController.getUserById, BookController.*, LoanController.libLoanRegister/libLoanReturn/libLoanQuery) só são alcançáveis dentro do menu exclusivo do papel LIBRARIAN (MainController.java:39-58), papel obtido do banco após a autenticação.

Ponto forte

Sem escalonamento de privilégio no autocadastro público

O fluxo de autocadastro acessível a qualquer pessoa não autenticada (UserController.singUp(), linhas 124-136, chamado a partir de MainController.startMenu() opção 2) fixa o papel como Role.COMMON diretamente no código-fonte, sem expor ao usuário nenhuma opção de escolher o papel LIBRARIAN. A opção de escolher o papel (UserController.libUserSingUp(), linha 52) só existe dentro do menu já restrito ao bibliotecário.

Categorias não aplicáveis à stack

2. Permissão definida no navegador — N/A

O LibManager é uma aplicação Java de linha de comando, executada como um único processo confiável (sem cliente web, sem API HTTP e sem separação front/back). A restrição de menus por papel (MainController.librarianMenu / commonUserMenu, MainController.java:34-37) não é uma barreira de UI que possa ser contornada "forjando uma requisição" — não existe canal de rede entre uma camada de apresentação desconfiada e o backend. Todo o fluxo roda no mesmo processo, com o papel do usuário lido do banco de dados logo após a autenticação (AuthDAO.login), nunca fornecido pelo lado "cliente". Categoria não aplicável à stack atual.

5. Inputs sem tratamento (XSS) — N/A

Não há renderização de HTML, JavaScript, templates de e-mail ou navegador envolvidos em nenhum ponto do sistema. Toda a entrada do usuário é lida via Scanner (InputUtils) e toda a saída é texto puro impresso via System.out no terminal (InterfaceUtils). Categoria não aplicável à stack atual (CLI Java puro).

Tabela de achados detalhados

Sev.	Arquivo:linha	Descrição
Crítica	V005__insert-users-admin.sql 1-4	**[F2] Credencial de administrador padrão fraca versionada no código-fonte**
Alta	LoanDAO.java 74-76	**[F1] Quebra de isolamento entre usuários na consulta de empréstimos**
Alta	UserDAO.java AuthDAO.java:15,19 · UserDAO.java:94-110,127-143	**[F3] Senhas armazenadas e comparadas em texto puro (sem hashing)**
Baixa	InputUtils.java 11-18	**[F4] Ausência de política mínima de senha**
Informativa	Diversos DAOs (ex.: AuthDAO.java:33-35, LoanDAO.java, BookDAO.java, UserDAO.java) catch(SQLException e){ e.printStackTrace(); }	**[F5] Tratamento de exceções via printStackTrace()**

Detalhamento arquivo por arquivo, linha por linha

[F1] Quebra de isolamento entre usuários na consulta de empréstimos

Alta

Arquivo: src/main/java/io/github/vitorfranca089/libmanager/dao/LoanDAO.java **Linhas:** 74-76

```
String sql = "SELECT * FROM loans INNER JOIN books ON books.id=loans.id_book " +
        "WHERE loans.id_user = ? AND loans.loan_status = 'BORROWED' OR loans.loan_status = 'OVERDUE' "
 +
        "ORDER BY loans.id ASC";
```

Descrição: O método findLoansByUser() é chamado tanto pelo menu do usuário comum ("Consultar empréstimos", com o próprio usuário autenticado) quanto pelo bibliotecário. O SQL monta a cláusula WHERE sem parênteses entre as condições combinadas com AND e OR. Como AND tem precedência maior que OR em SQL, a condição é avaliada como (loans.id_user = ? AND loan_status = 'BORROWED') OR (loan_status = 'OVERDUE'), e não como o pretendido loans.id_user = ? AND (loan_status = 'BORROWED' OR loan_status = 'OVERDUE'). Na prática, todo empréstimo com status OVERDUE de QUALQUER usuário é retornado, independentemente do parâmetro id_user.

Por que é explorável: Não exige nenhuma condição especial: qualquer usuário comum autenticado, ao acessar "Empréstimos > Consultar empréstimos" no seu próprio menu, já recebe na resposta os empréstimos atrasados (OVERDUE) de outros usuários — livro, datas de empréstimo/vencimento e ID do empréstimo — sem que o próprio usuário tenha relação com aquele empréstimo.

Impacto: Vazamento de dados entre contas (violação de isolamento/tenant), classificável como Broken Access Control (OWASP A01:2021 / CWE-284, CWE-639). Expõe metadados de outros usuários (quais livros estão atrasados e desde quando) e demonstra que a camada de dados não garante isolamento por usuário de forma confiável, quebrando uma garantia básica de privacidade do sistema.

Evidência de verificação dinâmica: Verificado experimentalmente em ambiente de teste local (dados fictícios): usuário fictício 'userA' com um empréstimo OVERDUE do livro 'Livro X' e usuário fictício 'userB' com um empréstimo BORROWED do livro 'Livro Y'. Ao logar como 'userB' e consultar "Meus empréstimos", a listagem retornada pela aplicação real incluiu o empréstimo atrasado de 'userA' junto com o próprio empréstimo de 'userB'. A mesma consulta, reescrita apenas adicionando parênteses ao redor da condição OR, retornou corretamente somente o empréstimo do usuário filtrado. Os dados de teste foram removidos do banco após a verificação.

Correção sugerida: Adicionar parênteses para restaurar a precedência lógica pretendida: WHERE loans.id_user = ? AND (loans.loan_status = 'BORROWED' OR loans.loan_status = 'OVERDUE')

[F2] Credencial de administrador padrão fraca versionada no código-fonte

Crítica

Arquivo: src/main/resources/db/migration/V005__insert-users-admin.sql **Linhas:** 1-4

```
INSERT INTO users
    (name, username, password, address, phone, role)
    VALUES
    ('admin', 'admin', '123', '', '', 'LIBRARIAN');
```

Descrição: A migração Flyway V005 é aplicada automaticamente em toda instalação nova do projeto (DatabaseConfig.migrateDatabase(), chamada em Main.java antes de abrir o menu) e cria, de forma incondicional, um usuário com o papel de maior privilégio do sistema (LIBRARIAN), usuário 'admin' e senha '123'. Não há nenhum mecanismo que force a troca dessa senha no primeiro acesso, nem validação de startup que alerte ou bloqueie o uso da credencial padrão.

Por que é explorável: Como o repositório é público, a credencial não é um segredo: está publicada no histórico do GitHub. Qualquer pessoa com acesso à aplicação (rodando localmente ou, futuramente, exposta em rede) pode logar diretamente como bibliotecário usando admin/123, sem qualquer etapa adicional.

Impacto: Comprometimento total do papel administrativo (LIBRARIAN): gestão de usuários (criar, editar, resetar senha de qualquer usuário para a senha padrão — UserController.libToDefaultPasswordUser()), gestão de livros e de todos os empréstimos do sistema. CWE-798 (Use of Hard-coded Credentials) / CWE-1188 (Insecure Default Initialization).

Correção sugerida: Não versionar credenciais reais mesmo como 'valor de fábrica'. Alternativas: (a) gerar uma senha aleatória forte no primeiro start e exigir troca obrigatória no primeiro login; (b) ler a senha inicial de uma variável de ambiente sem valor padrão, falhando o startup se ausente; (c) adicionar validação que rejeite login com a senha padrão sem antes forçar a troca.

[F3] Senhas armazenadas e comparadas em texto puro (sem hashing)

Alta

Arquivo: src/main/java/io/github/vitorfranca089/libmanager/dao/AuthDAO.java; src/main/java/io/github/vitorfranca089/libmanager/dao/UserDAO.java **Linhas:** AuthDAO.java:15,19 · UserDAO.java:94-110,127-143

```
// AuthDAO.java:15
String sql = "SELECT id, name, username, address, phone, role FROM users WHERE username = ? AND passwo
rd = ?";

// UserDAO.java:136 (verifyPassword)
return rs.getString(1).equals(oldPass);
```

Descrição: Em nenhum ponto do código (AuthDAO.login, UserDAO.updateUserWithPass, UserDAO.toDefaultPassword, UserDAO.verifyPassword) há aplicação de hashing (BCrypt, Argon2, PBKDF2) sobre a senha. O login compara a senha em texto puro diretamente na cláusula WHERE do SQL, e a troca/verificação de senha faz o mesmo com String.equals().

Por que é explorável: Exige acesso de leitura ao banco de dados (dump, backup, ou — combinado com o achado F2 — acesso administrativo já obtido) para expor as senhas de todos os usuários sem necessidade de quebra de hash.

Impacto: Qualquer vazamento de banco/backup expõe diretamente as senhas de todos os usuários em texto puro, permitindo reuso malicioso em outros sistemas (credential stuffing), já que reutilização de senha entre serviços é comum. CWE-256 (Plaintext Storage of a Password).

Correção sugerida: Adotar um algoritmo de hash de senha com salt (BCrypt/Argon2id), armazenar apenas o hash, comparar com verificação de hash (não com igualdade de string) e migrar as senhas existentes no próximo login bem-sucedido.

[F4] Ausência de política mínima de senha

Baixa

Arquivo: src/main/java/io/github/vitorfranca089/libmanager/util/InputUtils.java **Linhas:** 11-18

```
public static String getStringNotBlank(String prompt){
    String response;
    do{
        System.out.println(prompt);
        response = in.nextLine();
    }while(response.isBlank());
    return response;
}
```

Descrição: getStringNotBlank(), usado para capturar a senha no autocadastro (UserController.singUp()) e na edição de perfil, só rejeita valores vazios. Não há verificação de tamanho mínimo nem de complexidade, permitindo senhas como '1' ou 'a'.

Por que é explorável: Facilita ataques de força bruta/dicionário caso o sistema seja exposto em rede.

Impacto: Reduz a robustez geral da autenticação (CWE-521 - Weak Password Requirements).

Correção sugerida: Exigir tamanho mínimo (ex: 8 caracteres) e, idealmente, verificar contra listas de senhas comuns.

[F5] Tratamento de exceções via printStackTrace()

Informativa

Arquivo: Diversos DAOs (ex.: AuthDAO.java:33-35, LoanDAO.java, BookDAO.java, UserDAO.java) **Linhas:** catch(SQLException e){ e.printStackTrace(); }

```
}catch(SQLException e){
    e.printStackTrace();
}
```

Descrição: O tratamento padrão de SQLException em todas as camadas DAO apenas imprime o stack trace completo (incluindo, em alguns casos, fragmentos de SQL) na saída padrão, sem log estruturado nem tratamento diferenciado.

Por que é explorável: Hoje sem impacto direto, pois a saída vai para o terminal local do próprio usuário (aplicação CLI de processo único). Torna-se relevante caso o projeto evolua para uma API web/servidor, onde esse tipo de saída costuma vazar para logs acessíveis ou, por engano, para a resposta ao cliente.

Impacto: Baixo no cenário atual; risco de vazamento de informação técnica interna (CWE-209) se a arquitetura mudar para modo servidor/rede.

Correção sugerida: Substituir por logging estruturado (ex.: java.util.logging ou SLF4J) sem expor detalhes internos em qualquer saída voltada ao usuário final.

Recomendações priorizadas

P1 — Imediato

- Corrigir a precedência lógica em LoanDAO.findLoansByUser() (linhas 74-76), adicionando parênteses à condição OR (F1).
- Remover a senha fixa 'admin/123' da migração V005 e forçar definição/troca de senha no primeiro acesso administrativo (F2).
- Implementar hashing de senha (BCrypt/Argon2id) em AuthDAO e UserDAO, migrando os registros existentes (F3).

P2 — Curto prazo

- Adicionar teste automatizado cobrindo isolamento de dados entre usuários, para evitar regressão da falha de precedência lógica (F1).
- Definir política mínima de senha (tamanho e complexidade) na captura de entrada (F4).

P3 — Melhoria contínua

- Substituir e.printStackTrace() por logging estruturado sem exposição de detalhes internos, especialmente antes de qualquer evolução para modo servidor/API (F5).
- Revisar periodicamente o evento agendado do MySQL (V004) quanto à consistência com a lógica da aplicação.

Comunicação ao proprietário/mantenedor

Mantenedor: Vitor Miguel Mendes de França (autor e único mantenedor do repositório)

Canal: Comunicação direta ao mantenedor durante a própria sessão de auditoria (o mantenedor é integrante do grupo responsável por esta atividade), seguida do registro formal e rastreável através da abertura de Issues públicas no repositório GitHub do projeto.

Data: 20/09/2026

Evidências (links das Issues abertas):

- Issue 1 — F1, isolamento entre usuários: https://github.com/VitorFranca089/LibManager/issues/1
- Issue 2 — F2/F3, credenciais: https://github.com/VitorFranca089/LibManager/issues/2
- Issue 3 — F4/F5, hardening: https://github.com/VitorFranca089/LibManager/issues/3

Issues para o GitHub

Texto completo, em Markdown, pronto para copiar e colar na criação de Issues no GitHub. Achados triviais/relacionados do mesmo tema foram agrupados em uma única issue para evitar spam.

```
--- ISSUE 1 ---
Título: [Segurança] Isolamento quebrado: usuário comum vê empréstimos atrasados de outros usuários
Labels: security, severidade-alta

## Descrição do problema
O método `LoanDAO.findLoansByUser()` monta a cláusula `WHERE` sem parênteses entre as condições `AND`/`OR
`. Como `AND` tem precedência maior que `OR` em SQL, a condição vira `(id_user = ? AND status = 'BORROWED
') OR (status = 'OVERDUE')` em vez de `id_user = ? AND (status = 'BORROWED' OR status = 'OVERDUE')`. Isso
 faz a consulta retornar os empréstimos **OVERDUE de todos os usuários**, independentemente do `id_user`
filtrado — quebrando o isolamento entre contas.

## Evidência
`src/main/java/io/github/vitorfranca089/libmanager/dao/LoanDAO.java:74-76`

```java
String sql = "SELECT * FROM loans INNER JOIN books ON books.id=loans.id_book " +
        "WHERE loans.id_user = ? AND loans.loan_status = 'BORROWED' OR loans.loan_status = 'OVERDUE' " +
        "ORDER BY loans.id ASC";
```

Verificado em ambiente de teste local com dois usuários fictícios (`userA` com empréstimo OVERDUE, `userB
` com empréstimo BORROWED): ao logar como `userB` e consultar "Meus empréstimos", o empréstimo atrasado d
e `userA` aparece na listagem junto com o de `userB`.

Impacto
Vazamento de dados entre contas de usuários (Broken Access Control — OWASP A01:2021, CWE-284/CWE-639). Qu
alquer usuário comum autenticado enxerga quais livros outros usuários têm atrasados, desde quando, e o ID
 interno desses empréstimos.

Sugestão de correção
Adicionar parênteses para restaurar a precedência pretendida:

```sql
WHERE loans.id_user = ? AND (loans.loan_status = 'BORROWED' OR loans.loan_status = 'OVERDUE')
```

Critérios de aceite
- [ ] A consulta em LoanDAO.findLoansByUser() usa parênteses e só retorna empréstimos do id_user filtrado
.
- [ ] Teste automatizado (unitário/integração) cobre o cenário: dois usuários, um com empréstimo OVERDUE
e outro com BORROWED, validando que cada usuário só vê seus próprios empréstimos.
- [ ] Revisão manual confirmando que nenhuma outra query do projeto tem o mesmo padrão de AND/OR sem parê
nteses.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---
Título: [Segurança] Credencial admin padrão fraca (admin/123) e senhas em texto puro no banco
Labels: security, severidade-critica

Descrição do problema
Dois achados relacionados ao ciclo de vida de credenciais:

1. A migração Flyway `V005__insert-users-admin.sql` cria, em toda instalação nova, um usuário `admin` com
 senha `123` e papel `LIBRARIAN` (privilégio máximo), sem forçar troca de senha no primeiro acesso.
2. Nenhuma camada do projeto (`AuthDAO`, `UserDAO`) aplica hash sobre as senhas: elas são armazenadas e c
omparadas em texto puro (`WHERE ... password = ?`, `String.equals()`).

Evidência
`src/main/resources/db/migration/V005__insert-users-admin.sql:1-4`

```sql
```
```

```
INSERT INTO users
    (name, username, password, address, phone, role)
    VALUES
    ('admin', 'admin', '123', '', '', 'LIBRARIAN');
```

`src/main/java/io/github/vitorfranca089/libmanager/dao/AuthDAO.java:15`

```java
String sql = "SELECT id, name, username, address, phone, role FROM users WHERE username = ? AND password
= ?";
```

Impacto
A credencial padrão está publicada no histórico do repositório público, permitindo login administrativo i
mediato por qualquer pessoa com acesso à aplicação. Combinado com o armazenamento em texto puro, qualquer
 leitura do banco (dump/backup) expõe diretamente as senhas de todos os usuários (CWE-798, CWE-1188, CWE-
256).

Sugestão de correção
- Remover a senha fixa da migração; gerar senha aleatória no primeiro start ou lê-la de variável de ambie
nte obrigatória, forçando troca no primeiro login.
- Adotar hash de senha com salt (BCrypt/Argon2id) em `AuthDAO`/`UserDAO`, migrando os registros existente
s no próximo login bem-sucedido.

Critérios de aceite
- [ ] Nenhuma senha em texto puro é inserida via migration; o setup inicial força definição/troca de senh
a.
- [ ] AuthDAO.login() e UserDAO passam a comparar/armazenar hash (BCrypt/Argon2id), nunca a senha em text
o puro.
- [ ] Senhas já existentes no banco são migradas (hash aplicado) de forma transparente no próximo login.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---
Título: [Segurança] Hardening: política mínima de senha e logging de exceções JDBC
Labels: security, severidade-baixa

Descrição do problema
Dois itens de baixa severidade/hardening, agrupados por serem triviais de corrigir e não exigirem issue i
ndividual:

1. `InputUtils.getStringNotBlank()` só rejeita valores vazios — não há tamanho mínimo nem verificação de
complexidade para senhas.
2. Todo o tratamento de `SQLException` nas classes DAO usa apenas `e.printStackTrace()`, sem logging estr
uturado.

Evidência
`src/main/java/io/github/vitorfranca089/libmanager/util/InputUtils.java:11-18` (sem verificação de tamanh
o mínimo)

Padrão repetido em `AuthDAO.java`, `UserDAO.java`, `BookDAO.java`, `LoanDAO.java`:

```java
}catch(SQLException e){
    e.printStackTrace();
}
```

Impacto
Impacto baixo no cenário atual (aplicação CLI local), mas reduz a robustez da autenticação e pode vazar d
etalhes internos via stack trace caso o projeto evolua para um servidor/API exposto em rede (CWE-521, CWE
-209).

Sugestão de correção
- Exigir tamanho mínimo de senha (ex.: 8 caracteres) em `InputUtils`/fluxo de cadastro e troca de senha.
- Substituir `e.printStackTrace()` por logging estruturado (SLF4J/java.util.logging), sem expor detalhes

```
internos em saídas voltadas ao usuário final.

## Critérios de aceite
- [ ] Cadastro/troca de senha rejeita senhas abaixo de um tamanho mínimo definido.
- [ ] Exceções JDBC passam a ser tratadas por um logger estruturado em vez de printStackTrace().

--- FIM ISSUE 3 ---
```